

ACTL3142 Tutorial

Week 10: Unsupervised Learning

Nick Guo

School of Risk and Actuarial Studies, UNSW Sydney

T2 2026

Recap

From Week 9 (Moving Beyond Linearity):

- ▶ **Splines:** piecewise polynomials joined smoothly at knots. Smoothing splines penalise curvature instead of choosing knots.
- ▶ **GAMs:** extend splines to multiple predictors via additive structure and backfitting.
- ▶ Still **supervised**: we had a response Y and flexible ways to model $E[Y | X]$.

Fundamental shift: this week, there is **no response variable** Y . We only observe features $X_1, \dots, X_p$ and want to discover structure in the data.

Two main tools:

- ▶ **Clustering:** group similar observations together (K-Means, Hierarchical)
- ▶ **Dimension reduction:** compress many features into a few summaries (PCA)

Today: **K-Means & Hierarchical Clustering** and **Principal Component Analysis**.

K-Means Clustering: The Objective

Goal: partition n observations into K clusters so that points within each cluster are as similar as possible. Formally, minimise the total **within-cluster variation** (WCV):

$$\min_{C_1, \dots, C_K} \sum_{k=1}^K \underbrace{\sum_{i \in C_k} \sum_{j=1}^p (x_{ij} - \bar{x}_{kj})^2}_{\text{WCV in cluster } k}$$

Unpacking the sum:

- ▶ For each cluster k , compute its **centroid** $\bar{\mathbf{x}}_k$ (the mean of its points)
- ▶ For each point i in the cluster, its squared Euclidean distance to the centroid is $\sum_{j=1}^p (x_{ij} - \bar{x}_{kj})^2 = \|\mathbf{x}_i - \bar{\mathbf{x}}_k\|^2$
- ▶ Add these up across all points in the cluster, then across all K clusters

Intuition: we want tight, compact clusters. A good clustering has every point close to its centroid, so the total squared distance is small.

K-Means Clustering: The Algorithm

1. Randomly initialise K cluster centroids
2. Assign each observation to the **nearest** centroid
3. Recompute each centroid as the mean of its assigned points
4. Repeat steps 2–3 until assignments stop changing
5. Restart steps 1–4 many times, keep the solution with smallest total WCV

“Nearest” requires a **distance measure**. In this course we default to **Euclidean distance**, but other measures (Manhattan, simple matching, Gower’s) are useful in different settings. More on this shortly.

In R: `kmeans(x, centers = K)`

Local optima

K-means only guarantees convergence to a **local** minimum of WCV, not the global one. Different random starts can give different solutions, so step 5 is critical.

*Choose K via the **elbow method**: plot total WCV vs K and look for the bend where adding clusters gives diminishing returns.*

Hierarchical Clustering

Unlike K-means, **no need to choose K in advance**.

Algorithm (agglomerative / bottom-up):

1. Start: treat each of the n observations as its own cluster
2. Find the two closest clusters and merge them into one
3. Repeat step 2 until all observations belong to a single cluster

Output: a **dendrogram**, a tree that records the full merge history. Cut the tree at any height to obtain K clusters. Lower cut = more clusters.

In R: `hclust(dist(x), method = "complete")`

K-Means vs Hierarchical

K-means requires K upfront and only returns that single partition. Hierarchical clustering produces all possible K at once (via the dendrogram), so you can compare different cut heights after the fact.

*"How do we measure distance between two clusters?" This is the **linkage** choice, covered next.*

Linkage Types

When merging clusters, we need to define the distance between **two groups** of points. Different linkage methods give different dendrograms:

Linkage	Distance between A and B	Tends to produce
Complete	$\max_{a \in A, b \in B} d(a, b)$	Compact, balanced clusters
Single	$\min_{a \in A, b \in B} d(a, b)$	Extended, “chaining” clusters
Average	$\frac{1}{ A B } \sum_{a,b} d(a, b)$	Moderate balance
Centroid	$d(\bar{\mathbf{x}}_A, \bar{\mathbf{x}}_B)$	Can produce inversions

*Complete and average linkage tend to produce more balanced dendrograms. Centroid linkage can produce **inversions** (a merged cluster appearing below its children), making the dendrogram hard to interpret.*

Dissimilarity Measures

Linkage tells us how to measure distance between *clusters*. But first, we need a distance between *individual observations*. Common choices:

- ▶ **Euclidean:** $\sqrt{\sum_{j=1}^p (x_{ij} - x_{i'j})^2}$ (the default for most methods)
- ▶ **Manhattan:** $\sum_{j=1}^p |x_{ij} - x_{i'j}|$ (more robust to outliers in individual features)
- ▶ **Simple matching:** $\frac{1}{p} \sum_{j=1}^p I(x_{ij} \neq x_{i'j})$ (for categorical features)

The choice of distance metric matters as much as the choice of linkage. Always think about what “similar” means for your data. Euclidean treats all features equally, so standardise first if features are on different scales.

Q1: Dendrograms from a Dissimilarity Matrix

[Q1 screenshot placeholder: ISLR2 Q12.2]

PCA: Why Reduce Dimensions?

With many features (p large), the data is hard to visualise, expensive to compute with, and often noisy. If features are **correlated**, the information can be compressed into fewer dimensions.

Main uses of dimensionality reduction:

- ▶ **Visualisation**: project high-dimensional data onto 2D/3D to see structure
- ▶ **Preprocessing**: reduce noise and speed up downstream models (e.g. run PCA before fitting a classifier)
- ▶ **Compression**: store a compact representation of the data

Key idea: find new axes, called **principal components**, that capture the most variance in the data. The first PC is the direction along which the data varies the most; the second PC captures the most remaining variance orthogonal to the first, and so on.

PCA asks: "If I could only keep a few directions, which ones would preserve the most information?" Information here is measured by variance.

Q3: Motivations & Drawbacks of Dimensionality Reduction

[Q3 screenshot placeholder: HandsOnML3 Q8.1]

PCA: The First Principal Component

A **principal component** is a direction in feature space along which the data varies the most. Projecting the points onto that direction gives the largest possible variance.

Formally, the **first PC** Z_1 is the normalised linear combination of features with maximum variance:

$$Z_1 = \phi_{11}X_1 + \phi_{21}X_2 + \cdots + \phi_{p1}X_p, \quad \sum_{j=1}^p \phi_{j1}^2 = 1$$

- ▶ $\phi_1 = (\phi_{11}, \dots, \phi_{p1})^\top$: the **loading vector**, a unit-length direction in $\mathbb{R}^p$
- ▶ $z_{i1} = \phi_{11}x_{i1} + \cdots + \phi_{p1}x_{ip}$: the **score** for observation i (its projection onto ϕ_1)

The constraint $\sum \phi_{j1}^2 = 1$ prevents the variance from being inflated by simply scaling up the weights.

PCA: Subsequent PCs and Computation

The **second PC** is the direction of maximum variance that is **uncorrelated** with Z_1 (equivalently, its loading vector is orthogonal to ϕ_1). Each later PC is orthogonal to all earlier ones. Up to $\min(n - 1, p)$ PCs can be defined.

In R: `prcomp(x, scale = TRUE)`

- ▶ `$rotation`: loadings (one column per PC)
- ▶ `$x`: scores (one row per observation)
- ▶ `$sdev`: standard deviation along each PC

PCA $\neq$ linear regression

PCA minimises **perpendicular** distances from points to the PC line (all features treated symmetrically). Regression minimises **vertical** distances (prediction error, treating Y as special). Different objectives, different lines.

Always standardise (`scale = TRUE`) unless variables share units, otherwise the first PC just tracks whichever feature has the biggest scale.

PCA: Interpreting Loadings

The loading vector ϕ_1 tells you **which original variables drive each PC**:

- ▶ Large $|\phi_{j1}|$: variable X_j contributes heavily to PC1
- ▶ Small $|\phi_{j1}|$: variable X_j has little influence on PC1
- ▶ Sign indicates direction: positive means X_j increases with PC1

Geometric intuition (the Math Skill / Math Enjoyment example from the lecture):

- ▶ 2D data with correlated features $\rightarrow$ find the line of best fit through the cloud
- ▶ Project each point **perpendicularly** onto that line $\rightarrow$ the projections are PC1 scores
- ▶ The 1D representation (“Math Nerdiness”) captures most of the 2D variation
- ▶ The second PC is the orthogonal direction (captures the remaining spread)

Sign ambiguity

Each loading vector is unique only up to a **sign flip**: ϕ_1 and $-\phi_1$ define the same direction. Different software may report different signs. The scores flip correspondingly, but the variance explained is unchanged. ▶

PCA: Proportion of Variance Explained

Each PC captures some fraction of the total variance. The **proportion of variance explained** (PVE) by the m th PC is:

$$\text{PVE}_m = \frac{\sum_{i=1}^n \left(\sum_{j=1}^p \phi_{jm} x_{ij} \right)^2}{\sum_{j=1}^p \sum_{i=1}^n x_{ij}^2}$$

How many PCs to keep?

- ▶ **Scree plot:** plot PVE vs PC number, look for the “elbow”
- ▶ **Cumulative PVE:** choose the smallest M such that $\sum_{m=1}^M \text{PVE}_m \geq$ a threshold (e.g. 95%)

In R: `pve <- pr$sdev^2 / sum(pr$sdev^2); plot(cumsum(pve))`

*How quickly cumulative PVE rises depends on the correlation structure.
Highly correlated features (e.g. pixels in images) → a few PCs suffice.
Mostly independent features → need many PCs.*

Q6: PCA on a 1000-Dimensional Dataset

[Q6 screenshot placeholder: HandsOnML3 Q8.5]

Q4: Reversing Dimensionality Reduction

[Q4 screenshot placeholder: HandsOnML3 Q8.3]

Practical Considerations

Scaling:

- ▶ **Always standardise** before PCA or distance-based clustering when features are on different scales. Otherwise the method is dominated by whichever variable is largest.
- ▶ Exception: if all variables share the same units (e.g. gene expression levels), scaling may not be needed.

Validation and robustness (clustering):

- ▶ There is no single “correct” clustering. Results depend on the distance metric, linkage, and K .
- ▶ Try multiple methods. If the same groups emerge consistently, you can be more confident.
- ▶ Ask: do the clusters have a meaningful interpretation?

How many PCs / clusters?

- ▶ PCA: scree plot or cumulative PVE threshold (e.g. 90–95%)
- ▶ K-means: elbow method (total WCV vs K)
- ▶ Hierarchical: choose a cut height that gives interpretable groups

Unsupervised learning is inherently more subjective than supervised learning. There is no test set to tell you if your answer is “right.”
Domain knowledge matters.

Thanks for coming! Best of luck for your exams!